Case Study 2 — Interactive Prototype Specification

System:
Weber State University eWeber — Register for Classes (Add/Drop)

Step 1: Interaction Mechanics & Input Paradigm Mapping

1.1 User Flow Selection

I focused on searching for a class, adding it to a schedule, reviewing it, registering, and getting confirmation. In my previous audit, the biggest issue was that clicking Add didn't mean the student was registered. I wanted this version to make the pending and registered steps much more obvious.

The main interaction flow is:

Find Classes -> Search Results -> Add Class -> Review Schedule -> Register Classes -> Confirmation

In the original system, students could add a class to their schedule but still needed to select Submit to complete registration. I wanted my prototype to make this distinction more obvious.

The new interface should separate the process into three main steps: Find Classes, Review Schedule, and Confirmation. Classes are marked as pending until registration is completed, and the review screen gives users another chance to check their selections before submitting. Sort of like an online shopping cart.

1.2 Input Mechanics Analysis

The interface should work with a mouse, touchscreen, keyboard, and assistive technologies such as screen readers. Even though most students probably use a computer to register for classes, the design should not depend entirely on mouse interactions.

Interaction	Mouse/Touch	Keyboard/Screen Reader
Select subject	Click or tap dropdown	Tab to dropdown, arrow keys to select
Enter course number	Click or tap input field	Tab to field and type, screen reader announces label
Search classes	Click or tap search classes	Tab to button and press Enter or Space
Add class	Click or tap add class	Tab to button and press enter or space
Review schedule	Click or tap review classes	Tab to button and press enter or space
Remove class	Click or tap remove	Tab to button and press enter or space
Submit registration	Click or tap register selected classes	Tab to button and press enter or space

For touchscreen users, the interface mostly uses single taps rather than gestures such as swiping or dragging. I think this is important because requiring gestures could make the interface harder to use, especially for users who are unfamiliar with it.

Keyboard users should be able to move through the interface using Tab without needing a mouse. Screen-reader users should also be able to identify each control through its label and receive announcements when the schedule or registration status changes.

1.3 Fitts' Law and Touch Target Sizing

For my prototype, I am using a minimum of at least 44 × 44 pixels, with 48 × 48 pixels for primary actions. I am also keeping at least 8 pixels of spacing between actions to help stop accidental selections.

Interaction	Mouse/Touch	Keyboard/Screen Reader
Select subject	48 px height, full field width	12 px
Enter course number	48 px height, full field width	12 px
Search classes	Minimum 120 × 48 px	12 px

Add class	Minimum 100 × 48 px	8 px
Review schedule	Minimum 160 × 48 px	12 px
Remove class	Minimum 88 × 44 px	8 px
Submit registration	Minimum 200 × 48 px	12 px

These dimensions should make the websites actions easier to select on both desktop and touchscreen devices. Though with the design that I am planning, it is likely I will have even more spacing than the minimum I am requiring.

Step 2: High-Fidelity Prototyping & Micro-Interaction Design

2.0 Prototype Using Figma

Link:
CaseStudy2Prototype

2.1 Component State Definitions

For my registration prototype, I wanted the actions to provide clear feedback instead of leaving the user guessing if something worked. This is important because one of the problems I found in the original registration interface was that adding a class and actually registering for it were not clearly shown.

These are seven states for the actions on the interface. Not every action needs to use all seven states, but each state should have a consistent appearance and action for each state it is in.

State	Visual Appearance	Functional Behavior
Default	Normal button or input appearance with readable text and a clear border or background	Component is ready for interaction
Hover	Slight background or border color change	Indicates the pointer is over an interactive element
Focus	High-contrast outline around the selected	Shows keyboard users which element is currently selected

	control, at least 2 px thick	
Active	Button darkens slightly while being pressed	Provides immediate feedback that an input is being activated
Disabled	Muted appearance with an explanation nearby	Prevents an action until its requirements are met
Loading	Loading indicator and processing message	Prevents repeated submissions while an action is processing
Error	Visible error message and highlighted input or affected section	Explains what went wrong and how to fix it

I think the Focus and Loading states are really important. Without a clear focus indicator, keyboard users might lose track of where they are. Without loading feedback, users might click a button multiple times because they aren't sure if the system is loading or not.

2.2 Component State Matrix

This table shows how the seven states apply to the actions in the registration process.

Component	Default / Hover / Focus / Active	Disabled / Loading / Error
Subject Dropdown	Displays selected subject, hover changes border, focus shows outline, active opens options	Search cannot continue without a subject, missing selection displays inline error
Course Number Field	Accepts numbers, hover changes border, focus shows outline, active allows typing	Invalid input displays an error explaining that a valid course number is required
Search Classes Button	Normal purple button, hover darkens, focus shows outline, active shows pressed appearance	Loading displays "Searching classes…", failed search displays an error with retry guidance
Add Class Button	Shows Add Class, hover and focus provide feedback, active adds selection	Changes to "Added" after selection, unavailable or duplicate classes cannot be added
Remove Class Button	Removes a pending class, hover, focus, and	Disabled during registration processing,

	pressed states remain visible	errors explain if removal fails
Review Classes Button	Opens review screen when classes are selected	Disabled when no classes are selected, explanation tells user to add a class first
Register Selected Classes Button	Submits selected classes after review	Disabled with no selections, loading shows “Processing registration…”, errors provide recovery instructions
Back to Search Button	Returns to class search while preserving selections	Disabled during final submission to avoid interrupting processing
Start Another Example Button	Returns to the beginning after confirmation	Temporarily disabled while resetting the interface

For text fields, the Active state shows an interaction with the action. For buttons, the Active state should only appear while the user is pressing the button.

2.3 Micro-Interactions and Animation Timing

Micro-interactions are going to be small changes that show the users what the interface is doing. I wanted these effects to be noticeable without slowing down the registration page.

Interaction	**Animation / Feedback**	**Timing**
Button hover	Smooth background color transition	200 ms ease-out
Button press	Slight darkening or pressed appearance	100 ms
Keyboard focus	High-contrast outline appears immediately	0 ms
Adding a class	Schedule item appears with a short fade	250 ms ease-out
Removing a class	Schedule item fades out	200 ms ease-in
Changing registration steps	New screen fades into view	250 ms ease-in-out
Validation error	Error text appears directly below affected field	Immediate

Loading	Spinner or progress indicator appears with status text	Immediate when processing begins
Registration confirmation	Success message and registered classes appear	250 ms ease-out

I wanted to avoid long animations because students are usually trying to complete registration as quickly as possible.

2.4 Loading and Asynchronous Processing

One situation where loading feedback is really important is when a student clicks on Register Selected Classes. In a real registration system, the server might need time to check seat availability, prerequisites, schedule conflicts, and whether the student is eligible to register. So having some loading time here will not only give time for the server to do those things, but also give the user an indicator that it is loading.

If the system takes longer than expected to load, it should show feedback that registration is processing.

Review Classes -> Select Register -> Processing -> Success or Error

- The Register button changes to “Processing registration…”
- A loading indicator appears.
- Additional registration submissions are blocked.
- The selected classes remain visible so the student knows what is being processed.
- A status message announces that registration is being processed.

Once the loading finishes, the interface either displays the confirmation screen or explains why registration could not be completed. I want to make sure that a class isn’t shown as Registered before getting confirmation from the server. So instead the class should remain Pending until the system confirms success.

2.5 Error Handling and Recovery

Errors should explain what happened and what the user can do next. I wanted to avoid vague messages that require students to guess how to fix a problem.

Error Scenario	**Message / Recovery**
No subject selected	“Please select a subject before searching.”
Invalid course number	“Enter a valid course number using numbers only.”
No matching classes	“No classes matched your search. Try another subject or course number.”

Duplicate class selection	“This class is already in your schedule.”
No classes selected	“Add at least one class before reviewing your schedule.”
Registration failure	“Registration could not be completed. Your selected classes are still pending. Please try again.”
Class becomes unavailable	“This class is no longer available. Remove it or choose another section.”

Error messages should appear near the input or action instead of a popup as they do on the original website. They should also be communicated to screen-reader users through an announcement. For example, if a student enters an incorrect course number, the input should be flagged as invalid and the error should explain what needs to be changed. After fixing the information, the student should be able to retry without starting the entire process over.

2.6 State Transitions and Edge Cases

Adding the same class twice: The system should recognize that the class is already selected and prevent duplicate entries. The Add Class button can change to “Added” to make this obvious.

Removing the last selected class: The schedule should return to its empty state, the pending class count should update to zero, and the Review Classes button should become disabled.

Double-clicking Register: Once registration begins processing, the system should ignore additional activation attempts.

Registration fails after processing: The selected classes should stay in the review screen with their Pending status. The user should receive an explanation and be able to retry or make changes.

Registration succeeds: The interface should move to the Confirmation screen and display the registered classes. Pending labels should only change to Registered after success is confirmed.

I can see how thinking about these situations ahead of time is important because a static design might look fine how I have it now. But this will not only help me in the design process, but in the real world it would also help a development team understand what I want.

Step 3: Accessibility Architecture & WCAG Specification

This section documents the accessibility requirements for my prototype. My goal is to make the process easy for students who use a mouse, keyboard, touchscreen, screen reader, or voice controls. The Figma prototype demonstrates the layout, but below I'll try to explain the semantic structure, ARIA, and screen-reader announcements.

3.1 Semantic HTML Structure

The finished interface should use HTML instead of making every part of the page a generic div. This should give assistive technologies a good understanding of each screen and makes keyboard movements easier.

Screen / Area	Required Structure	Purpose
Overall page	<header>, <main id="main">, <footer>	Provide landmarks and a main-content destination for a skip link.
Progress indicator	<nav aria-label="Registration progress"> … </nav>	Identify the current step, set aria-current="step" on the active step.
Find Classes	<section aria-labelledby="searchHeading"> containing <form>, <fieldset>, <legend>, <label>, <select>, <input>, and <button>	Group related search controls and associate each input with a visible label.
Search results	<section aria-labelledby="resultsHeading"> with course headings and real <button> controls	Give results meaningful headings and accessible Add Class actions.
Schedule summary	<aside aria-labelledby="summaryHeading"> with selected classes and Review button	Distinguish pending selections from completed registration.
Review Schedule	<section aria-labelledby="reviewHeading"> with selected-class list and Back/Register buttons	Allow students to review and submit their selections.
Confirmation	<section aria-labelledby="confirmHeading"> with confirmed-class list and restart button	Clearly communicate that the demo registration process has finished.

Each screen should have a clear heading, and the inactive screens should be hidden both visually and from keyboard navigation.

3.2 ARIA Attributes and Dynamic Notifications

The ARIA should help and work with the HTML, not replace it.

Element / Event	Required Attribute or Behavior	Expected Result
Current progress step	aria-current="step"	Screen reader identifies the current stage.
Search form inputs	<label for="…">, aria-describedby for help/error text, aria-invalid="true" only when invalid	Input purpose and correction guidance are announced.
Search results update	role="status" or aria-live="polite"	Announce “3 classes found” without interrupting typing.
Class added or removed	aria-live="polite" on a dedicated status region	Announce “CS 1400 added. 1 class pending” or the updated count.
Invalid form submission	Visible inline error, linked with aria-describedby, alert region for important errors	Announce “Please select a subject before searching.”
Registration processing	role="status" / aria-live="polite", aria-busy="true" on processing region	Announce “Processing registration” while submission is in progress.
Registration success	Move focus to confirmation heading, announce status politely	Student hears that registration completed and can review results.
Expandable details, if included	aria-expanded="false" / "true" on the toggle, aria-controls pointing to panel ID	Communicate whether details are open or closed. Not needed for controls that do not expand.
Disabled review action	Native disabled button plus visible explanation, optionally keep an explanatory message nearby	User can understand why Review Classes is unavailable.

For errors, I would use an instant announcement only if an important action is needed, such as a failed registration submission. Updates like a class being added should use polite announcements so they do not interrupt other screen-reader speech.

3.3 Keyboard Navigation and Focus Management

Keyboard navigation should go through the actions in an order that makes sense to the user.

Stage	Expected Tab / Keyboard Sequence	Focus Handling
Find Classes	Skip to main content -> Subject -> Course number -> Search Classes -> result Add Class buttons -> Review Classes	Focus outline remains visible, Enter/Space activates buttons.
Review Schedule	Back to Search -> Remove buttons (where provided) -> Register Selected Classes	On opening review, move focus to “Review your classes” heading.
Confirmation	Start Another Example	On success, move focus to “Registration complete” heading.
Validation error	Return focus to first invalid field or error summary	Error is visible, announced, and associated with the relevant input.
Back to search	Return to prior search context	Keep selected classes and restore focus to a sensible search or review control.

Students should be able to use Tab to move between controls, arrow keys for dropdown choices, and Enter or Space to activate buttons. Focus should not move randomly while the user types or while results are being updated.

3.4 Example Screen-Reader Announcements

Interaction	Example Announcement
Subject field	“Subject, combo box, required. Choose a subject.”
Search completed	“2 classes found.”
Class selected	“CS 1400 added to your schedule. 1 class pending.”
Review opened	“Review your classes, heading level 2.”
Registration processing	“Processing registration. Please wait.”
Registration succeeded	“Registration complete — demo, heading level 2.”
Registration failed	“Registration could not be completed. Your classes are still pending. Try again.”

These are just some examples of what could be announced for certain actions.

3.5 Color Contrast and Visual Accessibility

WCAG 2.1 AA requires a contrast ratio of at least 4.5:1 for normal text and 3:1 for large text. Important non-text UI indicators, including visible control boundaries and focus indicators, generally need at least 3:1 contrast against adjacent colors.

Proposed Color Pair	Calculated Ratio	Relevant Threshold	Result
#492365 purple / #FFFFFF white	12.27:1	4.5:1 normal text	Pass
#252535 dark text / #FFFFFF white	15.06:1	4.5:1 normal text	Pass
#616273 secondary text / #FFFFFF white	6.0:1	4.5:1 normal text	Pass
#747482 muted text / #FFFFFF white	4.6:1	4.5:1 normal text	Pass

The ratios above are the calculations for the Figma Prototype.

3.6 Accessibility Verification Plan

To verify the final design, I would inspect all of my prototype controls for contrast and size. I would check that all labels are announced, the tab movement matches the visual flow, focus moves to the new screen, status updates are spoken once, and errors explain how to fix the issue.

Overall, these requirements directly address problems I identified in the original registration interface, especially unclear registration status, difficult keyboard access, error feedback that was easy to miss, and contrast that made certain labels nearly impossible to read. The goal is for students to understand exactly where they are in the process and what action they need to take next.

Step 4: Post-mortem reflection

1. How did defining component states help?

A static picture can show where a button goes, but not what happens after someone clicks it. Making this prototype forced me to think about hover, keyboard focus, validation, and what should happen while something is processing. For example, I made

sure Register cannot be pressed repeatedly while it is working. I think those are things that could easily get missed when only looking at a wireframe.

2. How does latency feedback prevent repeated submissions?

When someone clicks Register selected classes, the prototype waits before showing confirmation. Without feedback, the user might click again because they think nothing happened. I changed the button to “Registering…”, disabled it, and added a screen-reader announcement. Once finished, the page clearly says Registered. That makes it easier to trust that the action went through.

Repository and Prototype Links

Github:
CaseStudy2Repo

Prototype:
CaseStudy2Prototype